

DRONE SWARM

1. Introduction

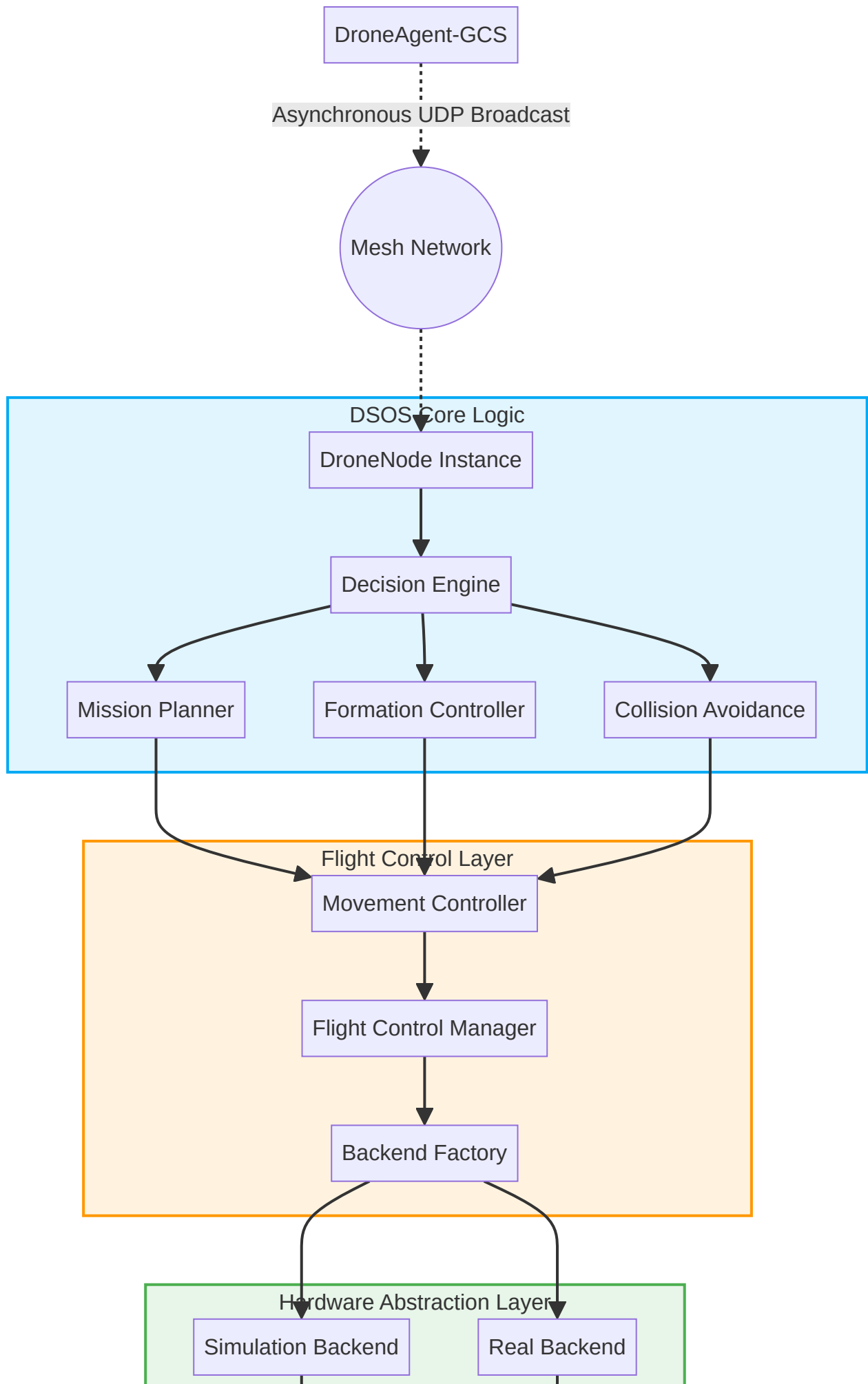
The paradigm of multi-UAV (Unmanned Aerial Vehicle) operations has traditionally relied on centralized, leader-follower architectures. In these systems, a single drone or Ground Control Station (GCS) dictates the behavior of the swarm. While simpler to implement, leader-follower systems present significant limitations: they suffer from a single point of failure, severe scalability bottlenecks due to bandwidth constraints at the leader node, and high latency in dynamic environments where rapid local decision-making is essential.

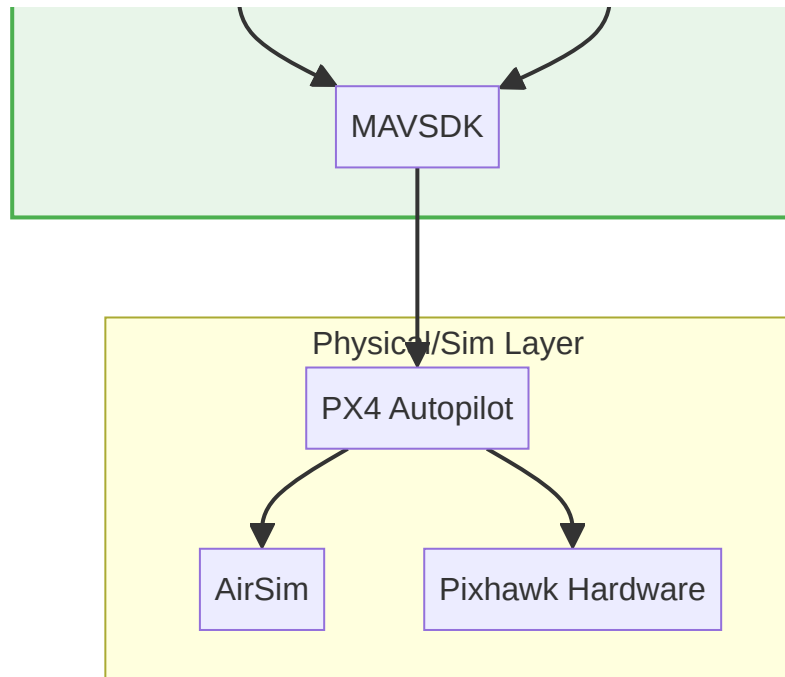
This project introduces a **Fully Decentralized Drone Swarm Operating System (DSOS)**. By eliminating the leader node, every drone in the swarm executes an identical, autonomous software stack (`DroneNode`). Decentralized systems are inherently more scalable because decision-making is distributed, relying only on peer-to-peer (mesh) data exchanges. If a node fails, the swarm mathematically reorganizes itself without halting the mission, achieving true fault tolerance and deterministic scalability.

2. Complete System Architecture

The DSOS architecture is purposefully stratified to decouple high-level mission logic from low-level flight hardware, allowing seamless execution on both simulation environments and real hardware.

Overall Architecture Diagram





Software Layer Diagram

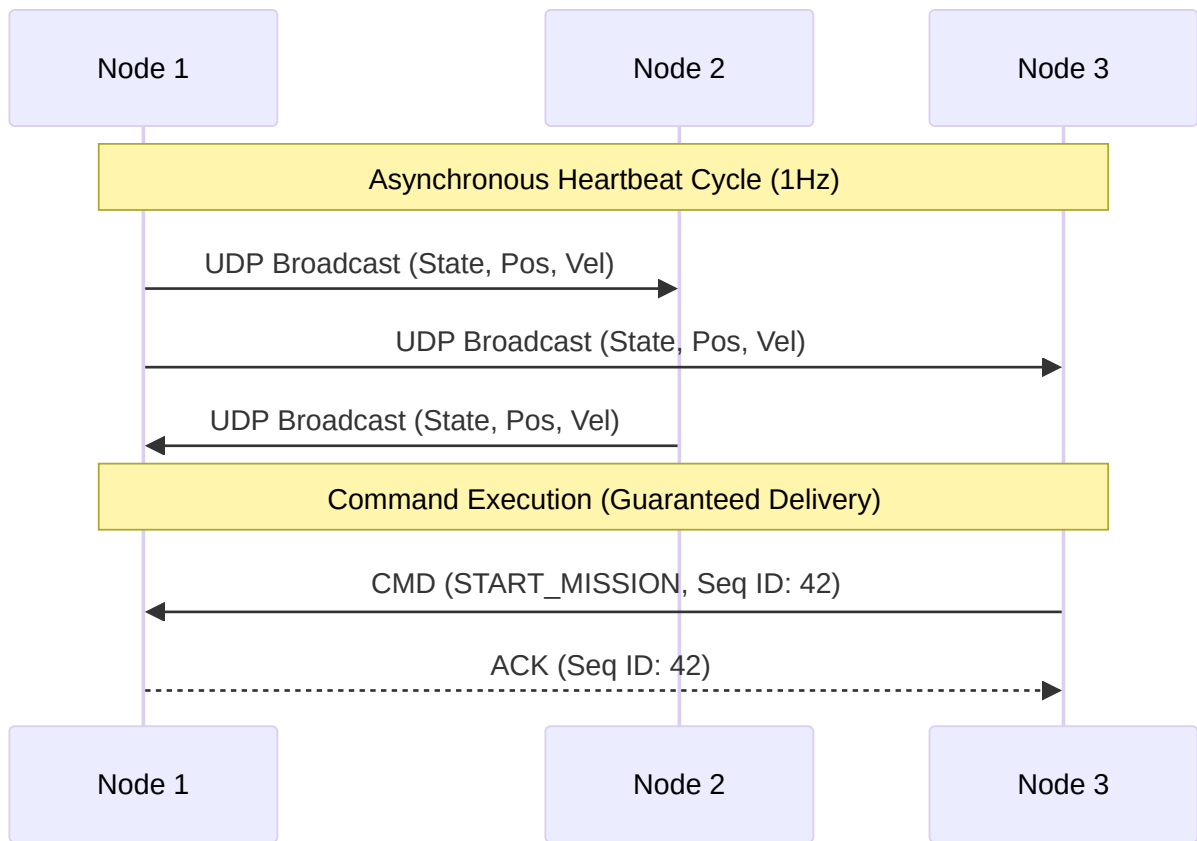


Module Explanations:

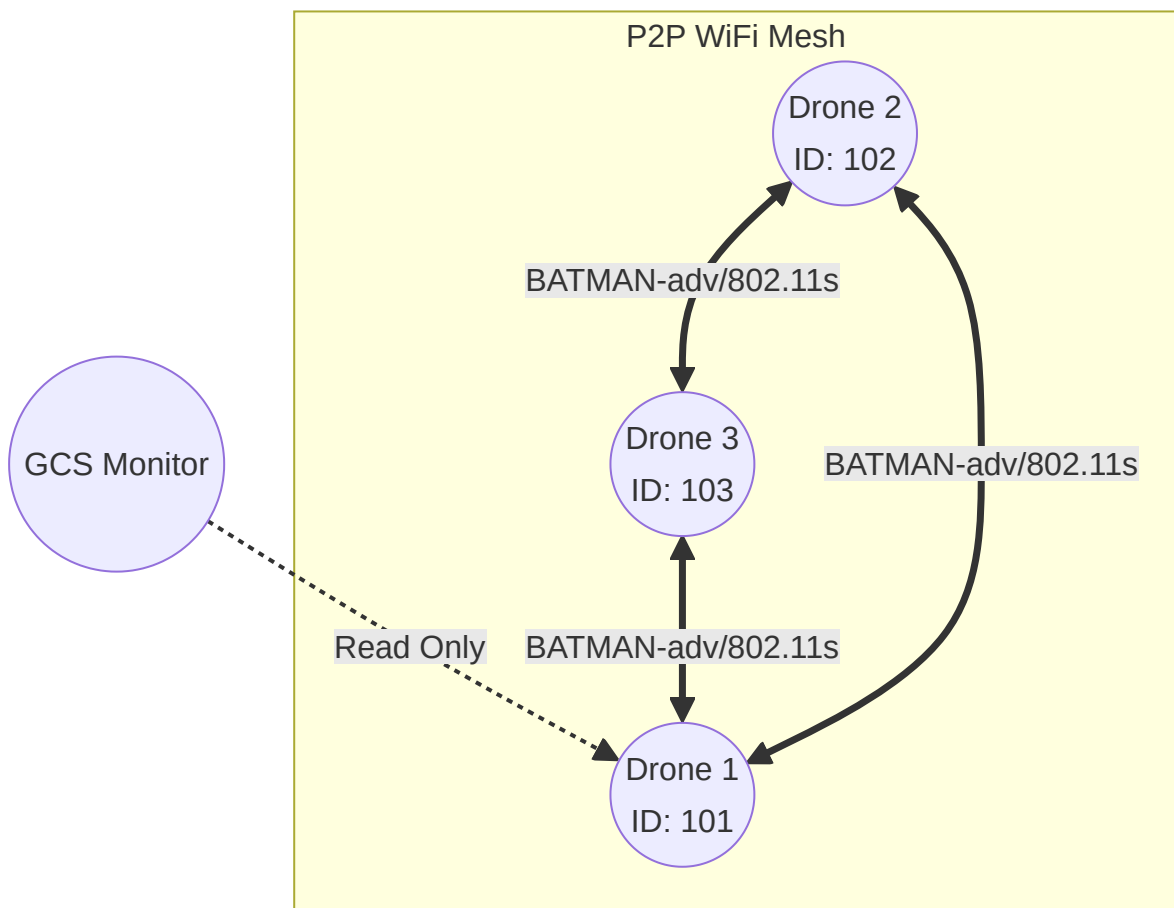
- **DroneAgent-GCS:** Acts strictly as a non-essential human interface to inject mission parameters (e.g., "Scan Area") into the mesh. It has no continuous control authority.
- **DroneNode:** The main process on every UAV, instantiating the submodules and managing the fixed-frequency control loop (50Hz).
- **Decision Engine & Mission Planner:** Evaluates mesh heartbeats and internal states to select behaviors (e.g., transition from IDLE to TAKEOFF).
- **Formation & Collision Avoidance:** Generates target velocity vectors based on neighbor proximity and desired geometric slots.
- **Movement Controller & Flight Control Manager:** Blends vectors from Mission, Formation, and Collision avoidance into a final bounded NED (North-East-Down) velocity command.
- **Backend Factory:** Polymorphically switches between `SimulationBackend` and `RealBackend` to translate standardized DSOS commands into MAVSDK API calls.

3. Decentralized Swarm Methodology

Communication Diagram



Three-Drone Mesh Diagram



Key Mechanisms:

- **Absolute Decentralization:** The system strictly separates the **DroneAgent-GCS** (which only acts as a passive observer or high-level task issuer) from the **DroneNode** (the autonomous brain running on every single drone). If the GCS crashes or disconnects, the swarm continues its mission unaffected because all decision-making logic resides inside the local **DroneNode** instances.
- **Neighbor Discovery:** Drones blindly broadcast **HEARTBEAT** packets over UDP on a shared port. When a drone hears a new ID, it adds it to its internal **NeighborTable**.
- **Heartbeat & Mesh Communication:** Every drone broadcasts its state (Latitude, Longitude, Altitude, Velocity, Battery, State) at 1Hz.
- **Distributed Decision Making & Task Allocation:** Consensus is reached via distributed averaging and bidding. If a waypoint must be visited, the drone with the lowest cost (distance + battery penalty) assigns the task to itself.
- **Failure Recovery:** If a neighbor's heartbeat is lost for >5 seconds, it is marked dead. The remaining swarm instantly recalculates formation slots to maintain symmetry without requiring a "leader" to reorganize them.

3.2 Formal Algorithmic and Mathematical Mapping

To prove true decentralization, the exact software logic running on the **DroneNode** maps directly to established mathematical control models. The architecture guarantees collision-free flight and formation stability with zero continuous input from **DroneAgent-GCS**.

Algorithm 1: Distributed State Consensus (Decision Engine)

To operate without a leader, the **DroneNode** runs a distributed averaging consensus algorithm continuously. For a state variable x_i (such as the agreed swarm centroid):

$$x_i(t+1) = x_i(t) + \epsilon \sum_{j \in N_i(t)} a_{ij}(x_j(t) - x_i(t))$$

- $N_i(t)$: The local **NeighborTable** updated asynchronously via UDP heartbeats.
- a_{ij} : Adjacency weight (1 if connected, 0 otherwise).
- ϵ : Consensus convergence rate (bounded by $0 < \epsilon < 1/\Delta_{max}$).

```
Function UpdateConsensus(local_state, neighbor_table):  
    sum_diff = 0  
    For each neighbor in neighbor_table:  
        If time_since_last_heartbeat(neighbor) < T_timeout:  
            sum_diff += (neighbor.state - local_state)  
    local_state = local_state + (epsilon * sum_diff)  
    Return local_state
```

Algorithm 2: Virtual Structure Formation (Formation Controller)

Instead of following a leader drone, drones follow a virtual centroid C_i agreed upon in Algorithm 1. The desired position $P_{desired,i}$ for a **DroneNode** with a locally assigned geometric slot offset O_i is:

$$P_{desired,i} = C_i + R(\theta) \cdot O_i$$

The corrective velocity is calculated via a Proportional-Derivative (PD) controller:

$$V_{formation,i} = K_p(P_{desired,i} - P_{current,i}) + K_d \frac{d}{dt}(P_{desired,i} - P_{current,i})$$

```
Function CalculateFormationVelocity(centroid, my_slot, current_pos, current_vel)
    P_desired = centroid + rotate(my_slot, swarm_heading)
    position_error = P_desired - current_pos
    velocity_error = (position_error - previous_error) / dt
    V_formation = (Kp * position_error) + (Kd * velocity_error)
    previous_error = position_error
    Return V_formation
```

Algorithm 3: Reactive Collision Avoidance (Artificial Potential Fields)

To prevent mid-air collisions without central coordination, an APF generates a repulsive force $F_{rep,ij}$ from neighbor j :
$$F_{rep,ij} = \begin{cases} k_{rep} \left(\frac{1}{d_{ij}^2} - \frac{1}{D_{safe}^2} \right) \frac{1}{d_{ij}^2} & \text{if } d_{ij} < D_{safe} \\ 0 & \text{if } d_{ij} \geq D_{safe} \end{cases}$$

$$V_{collision,i} = \sum_{j \in N_i(t)} F_{rep,ij}$$

```
Function CalculateAvoidanceVelocity(current_pos, neighbor_table):
    V_col = [0, 0, 0]
    For each neighbor in neighbor_table:
        distance = norm(current_pos - neighbor.pos)
        If distance < D_safe:
            direction_vector = normalize(current_pos - neighbor.pos)
            magnitude = K_rep * ((1/distance) - (1/D_safe)) * (1/(distance^2))
            V_col += direction_vector * magnitude
    Return V_col
```

Algorithm 4: Flight Control Manager (Vector Blending)

The final step in the software loop blends these mathematically derived vectors into a single physical command sent to the MAVSDK backend:

$$V_{final} = W_{miss}V_{mission} + W_{form}V_{formation} + W_{col}V_{collision}$$

```
Function ExecuteFlightControl(V_miss, V_form, V_col, distance_to_closest_obstacle)
    If distance_to_closest_obstacle < D_critical:
```

```

W_col = 1.0; W_form = 0.0; W_miss = 0.0 // Emergency override
Else:
W_col = exp(-alpha * (distance_to_closest_obstacle - D_safe))
W_form = 1.0 - W_col
W_miss = 1.0 - W_col

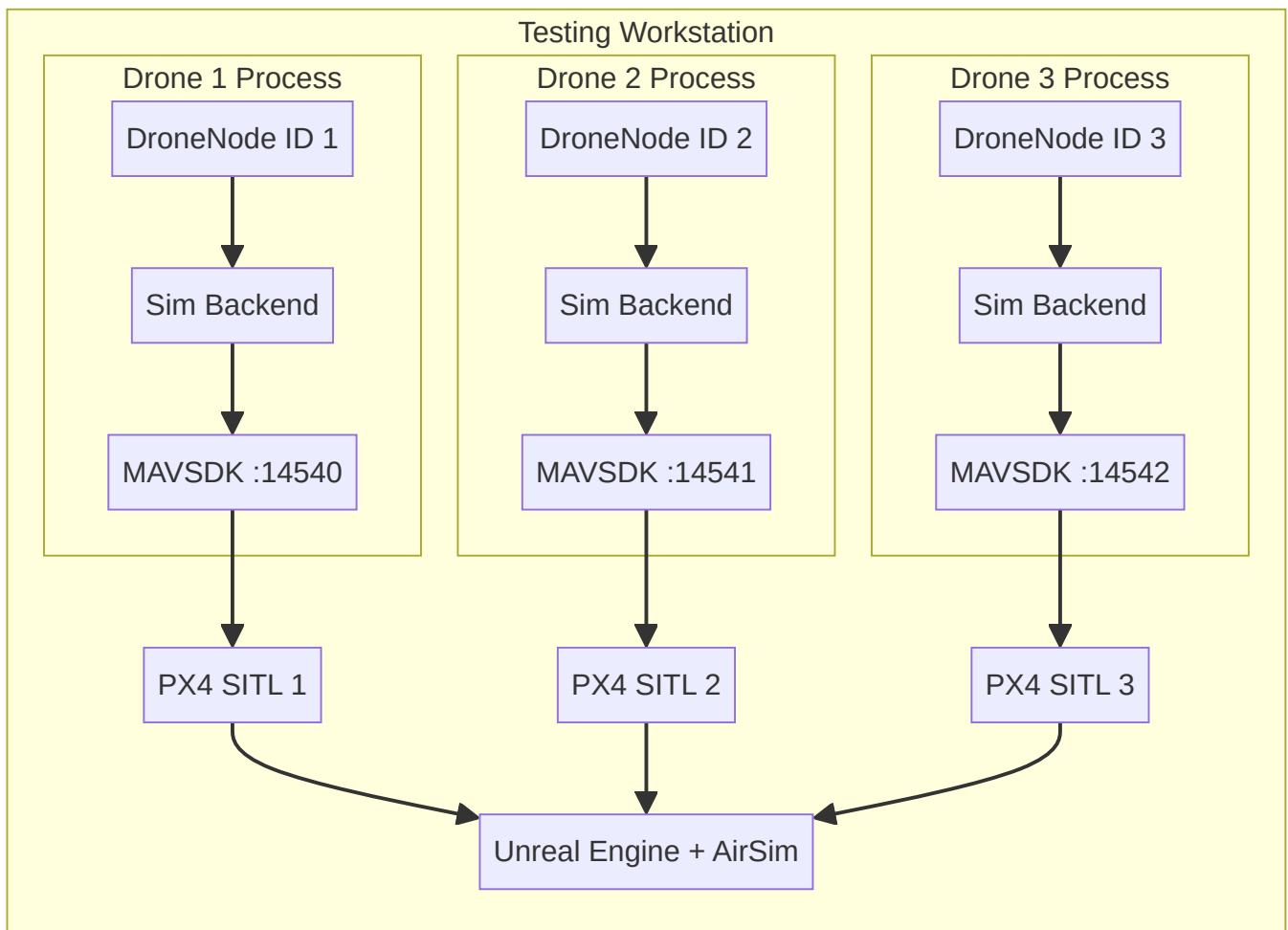
V_final = (W_miss * V_miss) + (W_form * V_form) + (W_col * V_col)
V_final = clamp_to_hardware_limits(V_final, MAX_SPEED)
MAVSDK.set_velocity_ned(V_final)

```

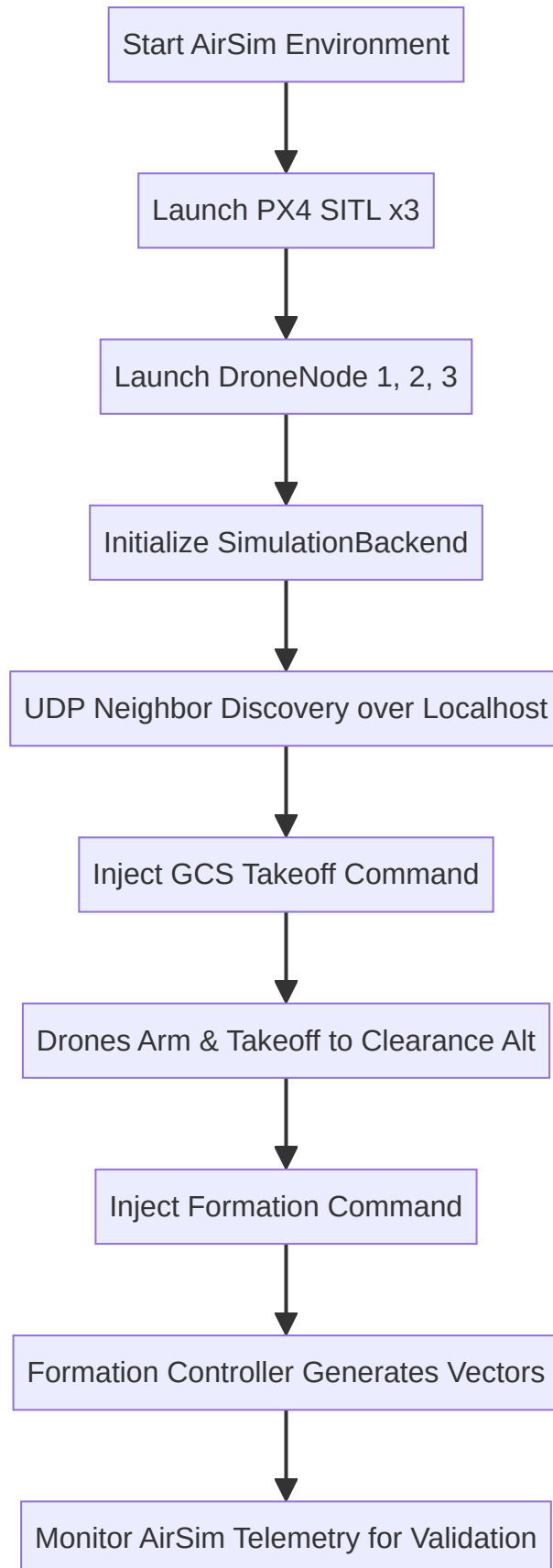
4. AirSim Three-Drone Testing Methodology

Testing in simulation is functionally identical to real-world deployment due to the **BackendFactory**.

AirSim Testing Diagram



AirSim Workflow



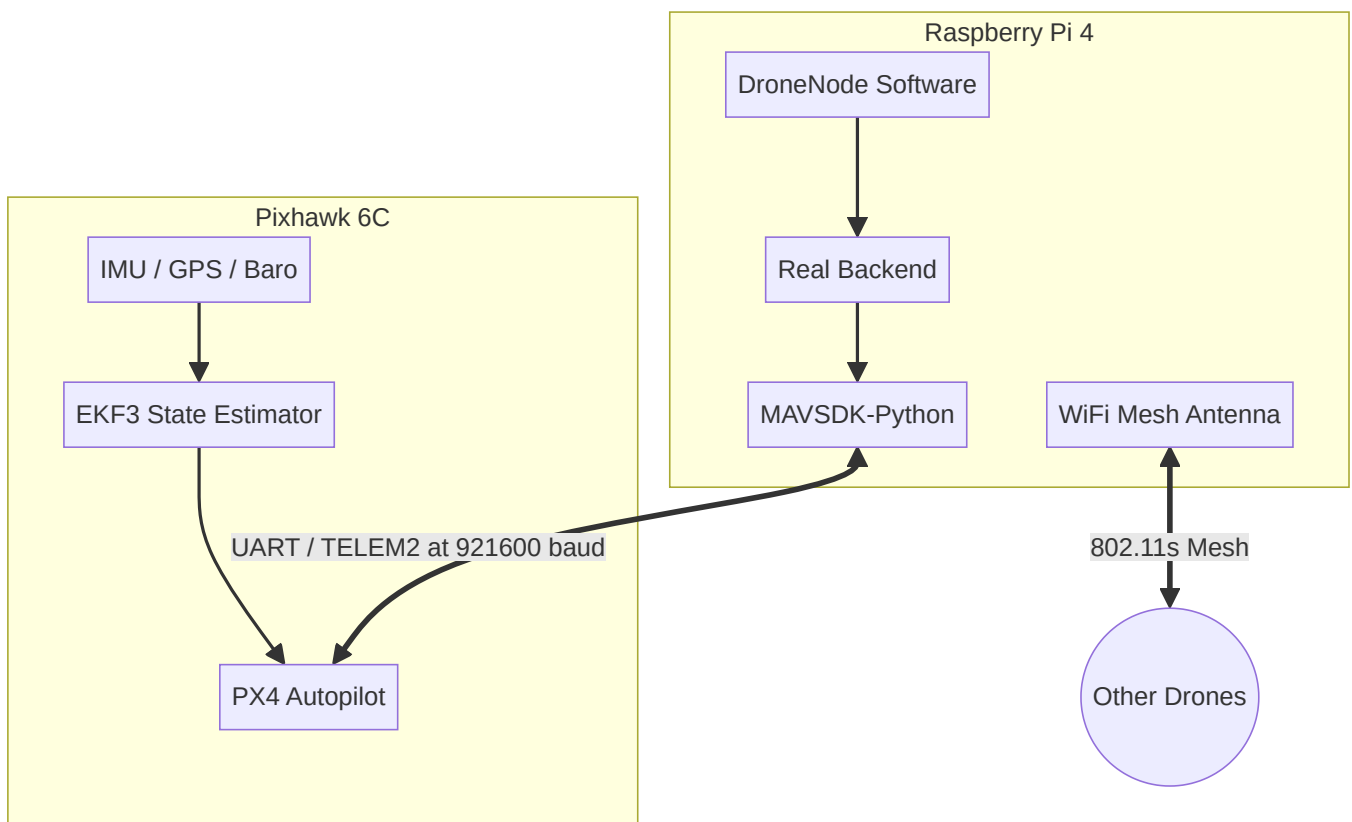
Testing Procedure:

1. **PX4 SITL & AirSim:** Spin up multiple instances of PX4 SITL connecting to a single Unreal Engine AirSim instance.

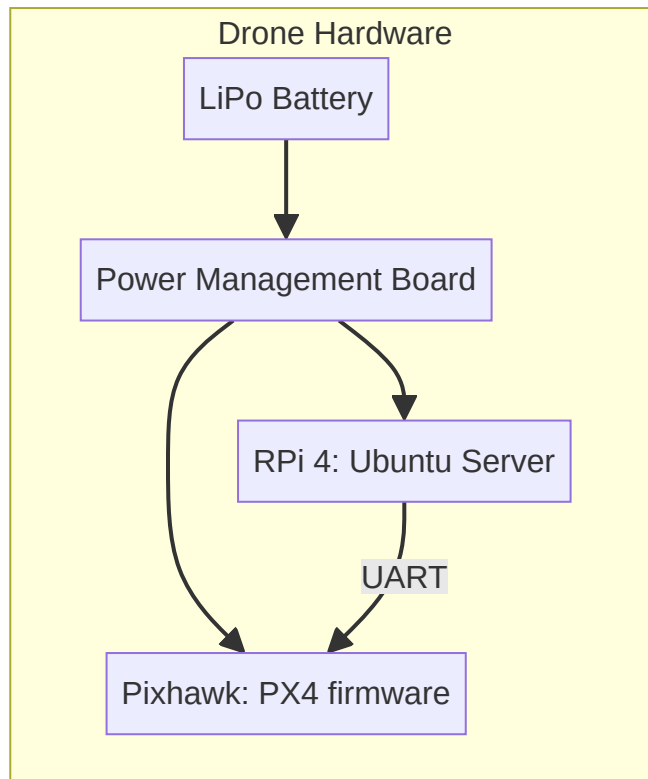
2. **MAVSDK Binding:** Each **DroneNode** is configured with a different UDP port mapping (e.g., 14540, 14541).
 3. **Takeoff & Movement:** Validate that local FSMs transition properly and that Offboard velocity control commands yield smooth simulated trajectories.
 4. **Formation & Collision:** Intentionally command drones to cross paths to validate APF repulsive vectors in 3D space.
-

5. Real Drone Deployment Methodology

Raspberry Pi + Pixhawk Diagram



Deployment Diagram



Real Drone Workflow

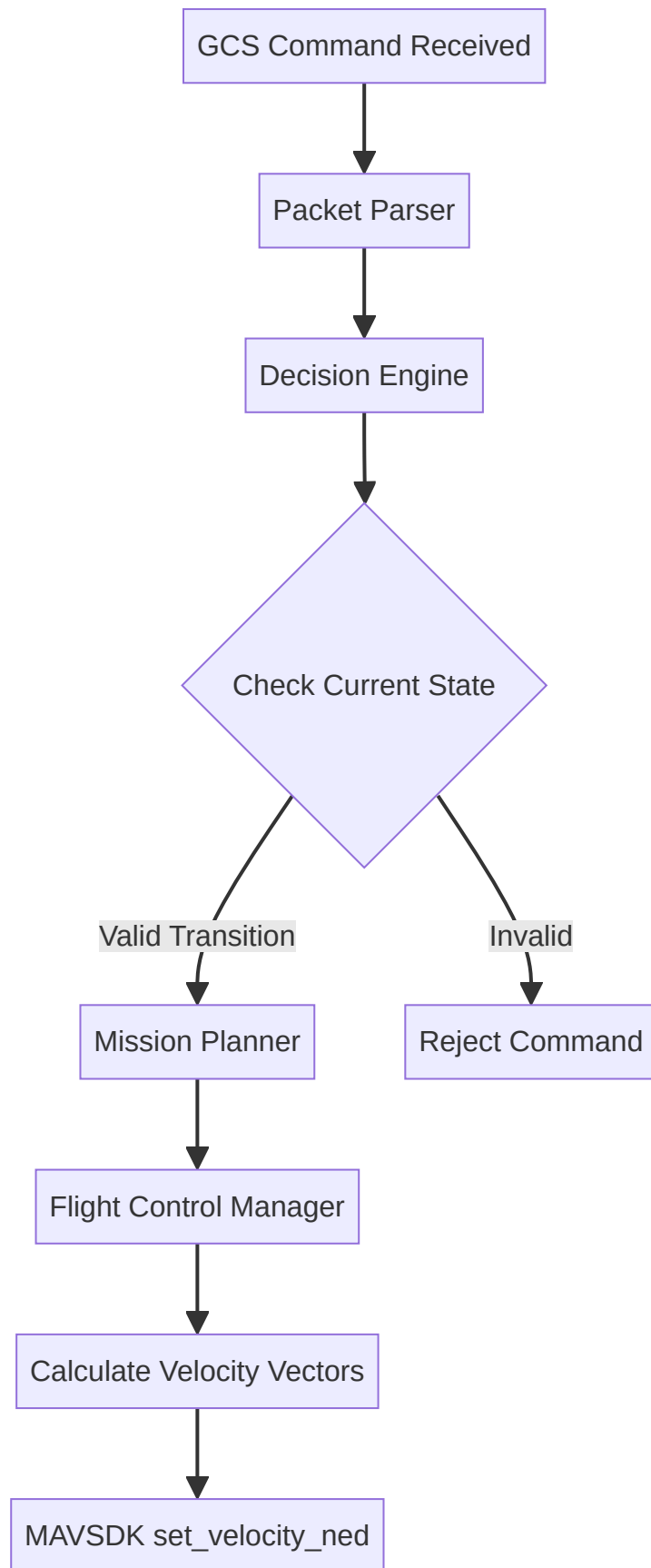


Deployment Specifics:

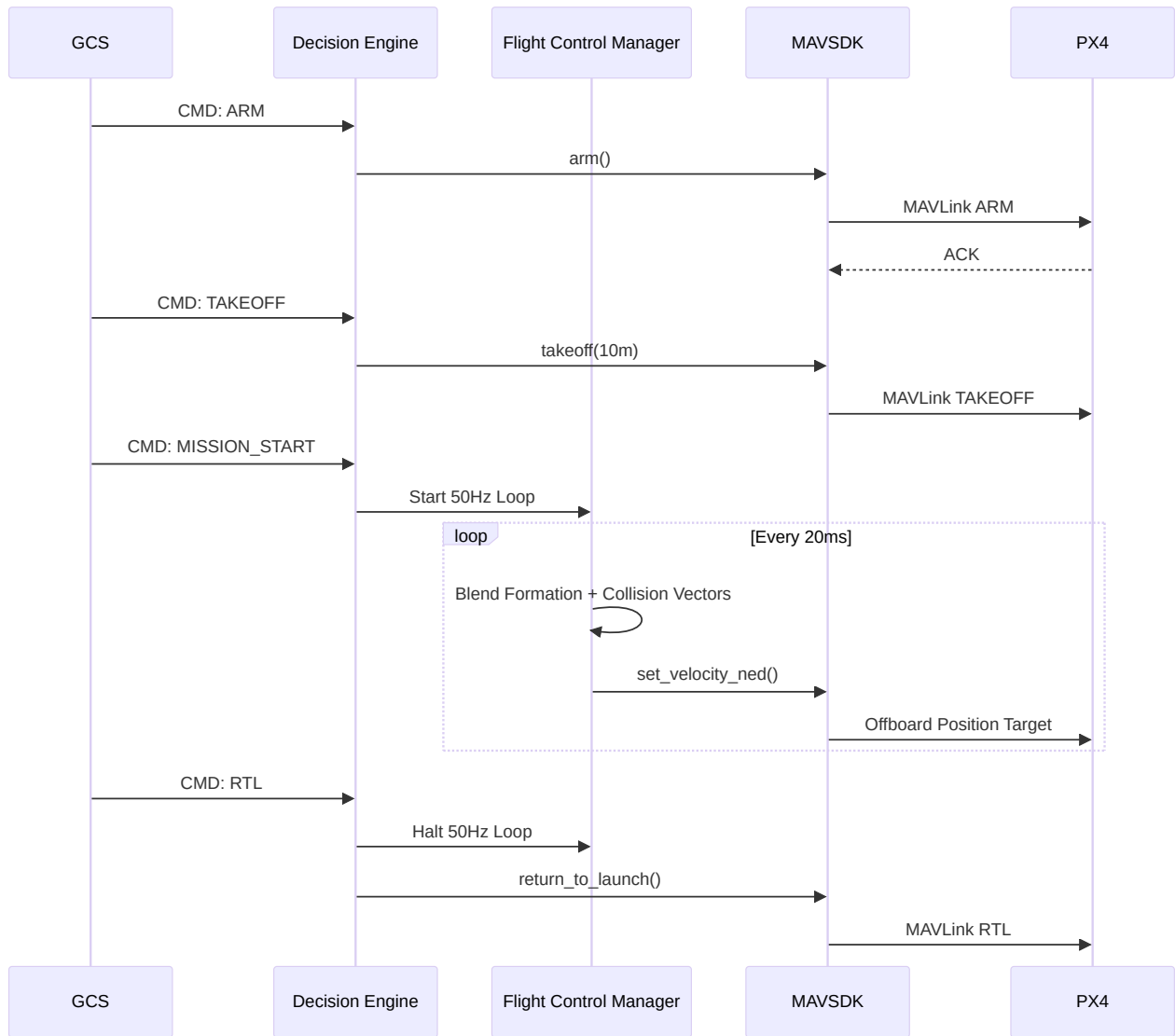
- **UART/USB Setup:** `RealBackend` explicitly connects to a serial port (e.g., `/dev/ttyTHS1`) rather than a UDP port.
- **Safety Checks:** The `RealBackend` enforces checks for GPS 3D Fix, low battery voltage, and EKF variance before allowing Arming.
- **Backend Switching:** This is handled transparently via a JSON config file read at boot, requiring zero code changes in the autonomy stack.

6. Runtime Command Flow

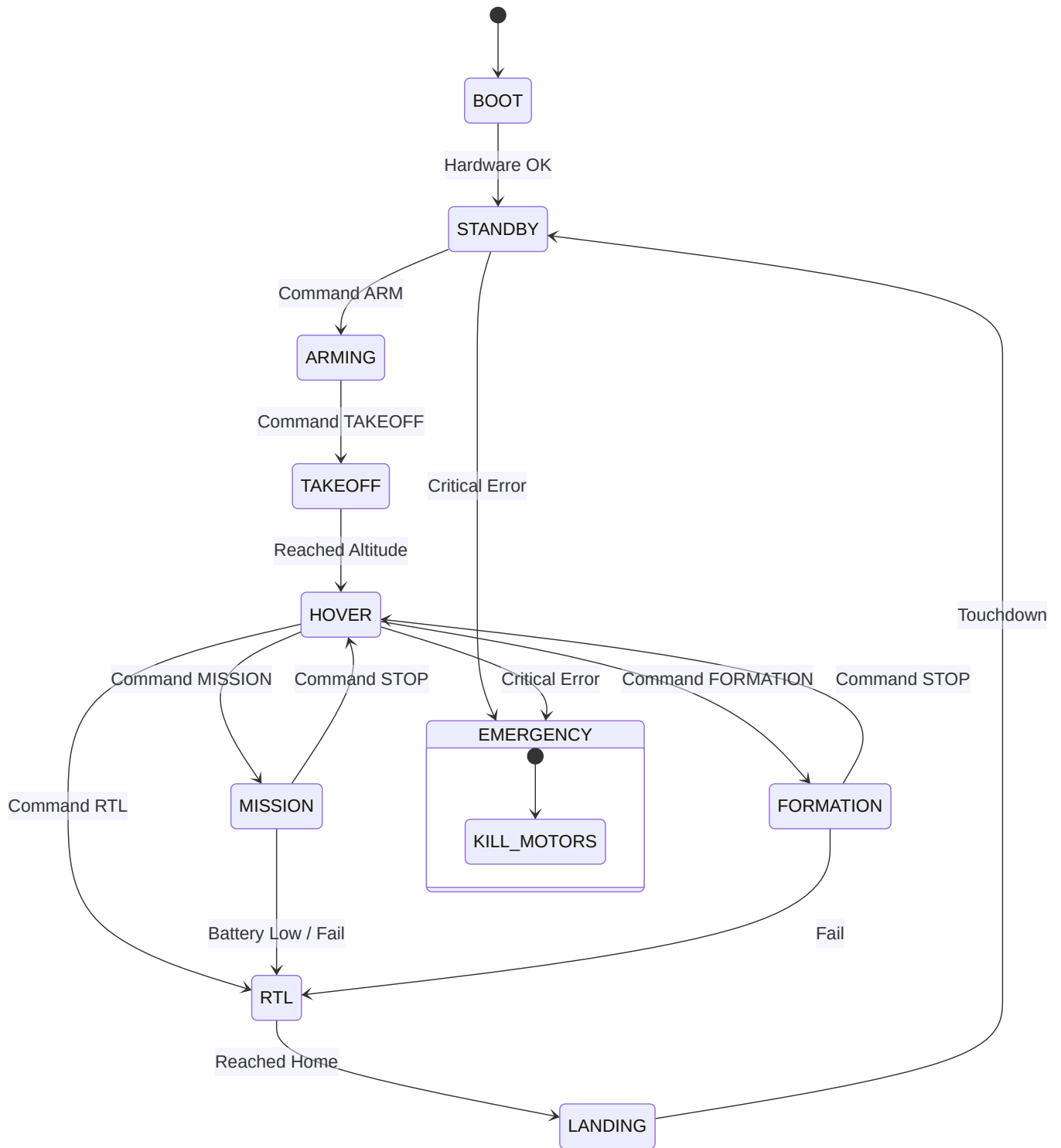
Runtime Flow Diagram



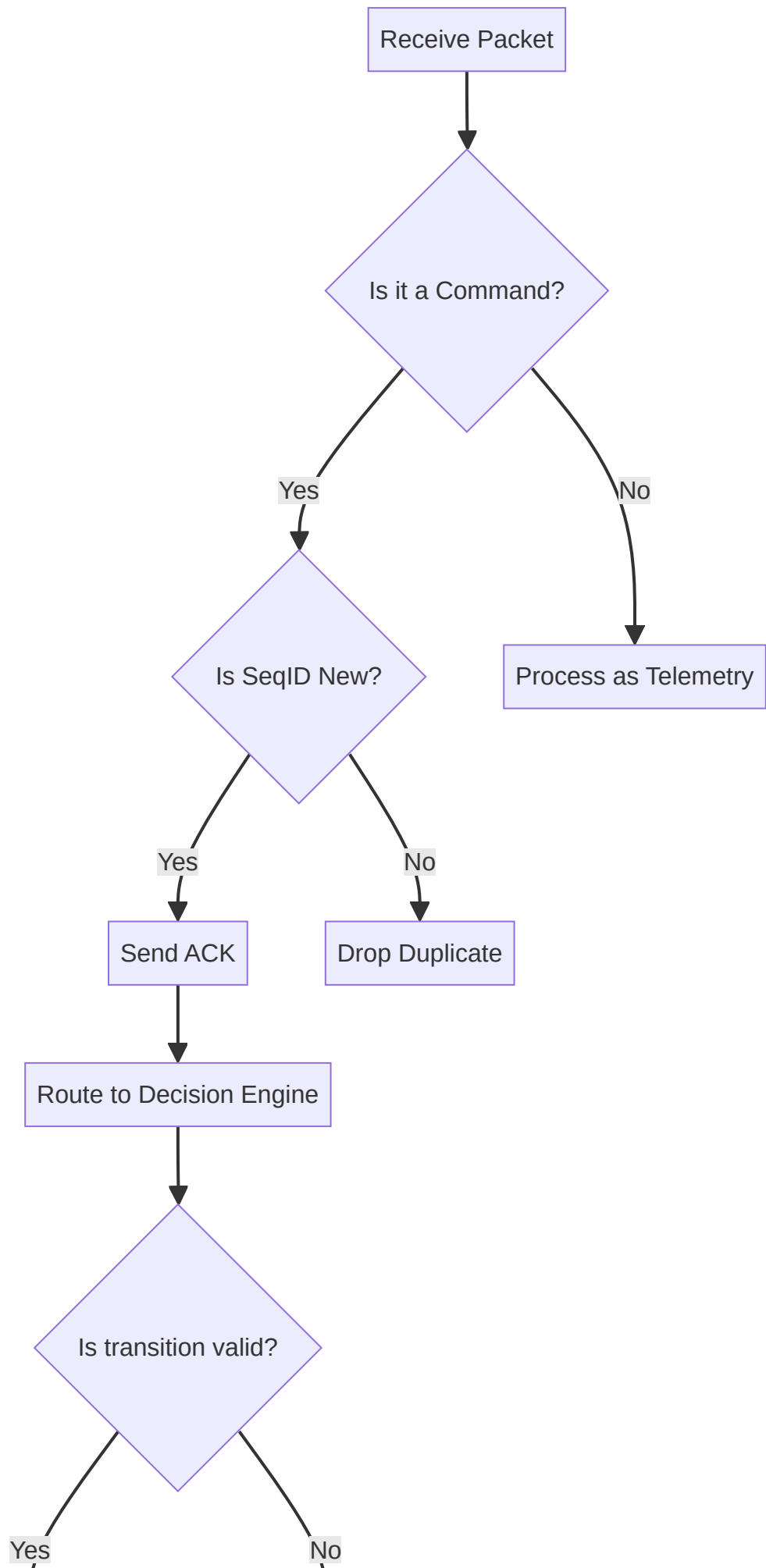
Sequence Diagram

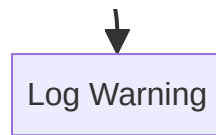
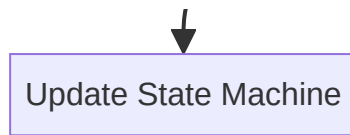


State Diagram

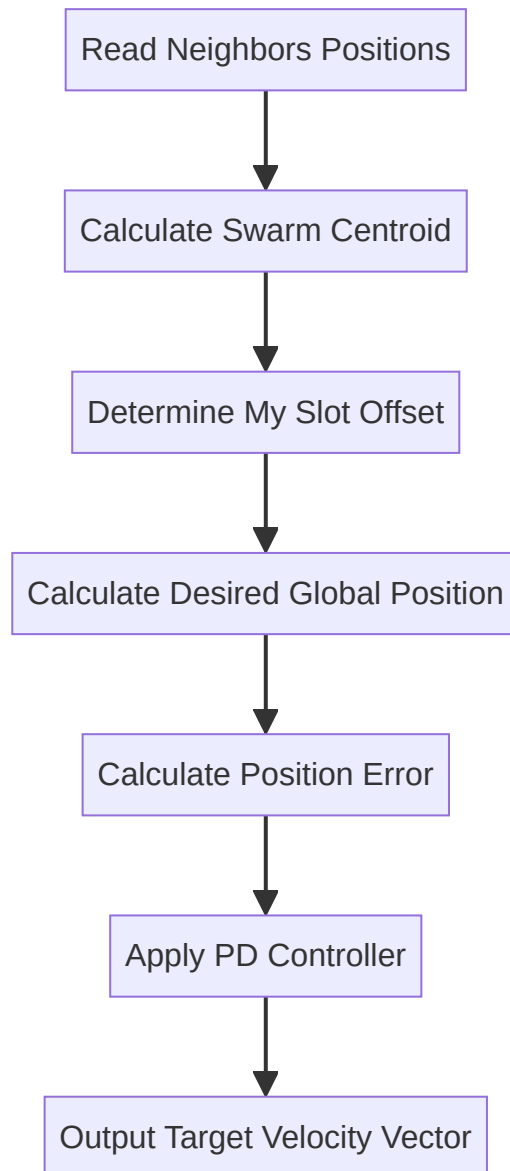


Command Execution Flowchart

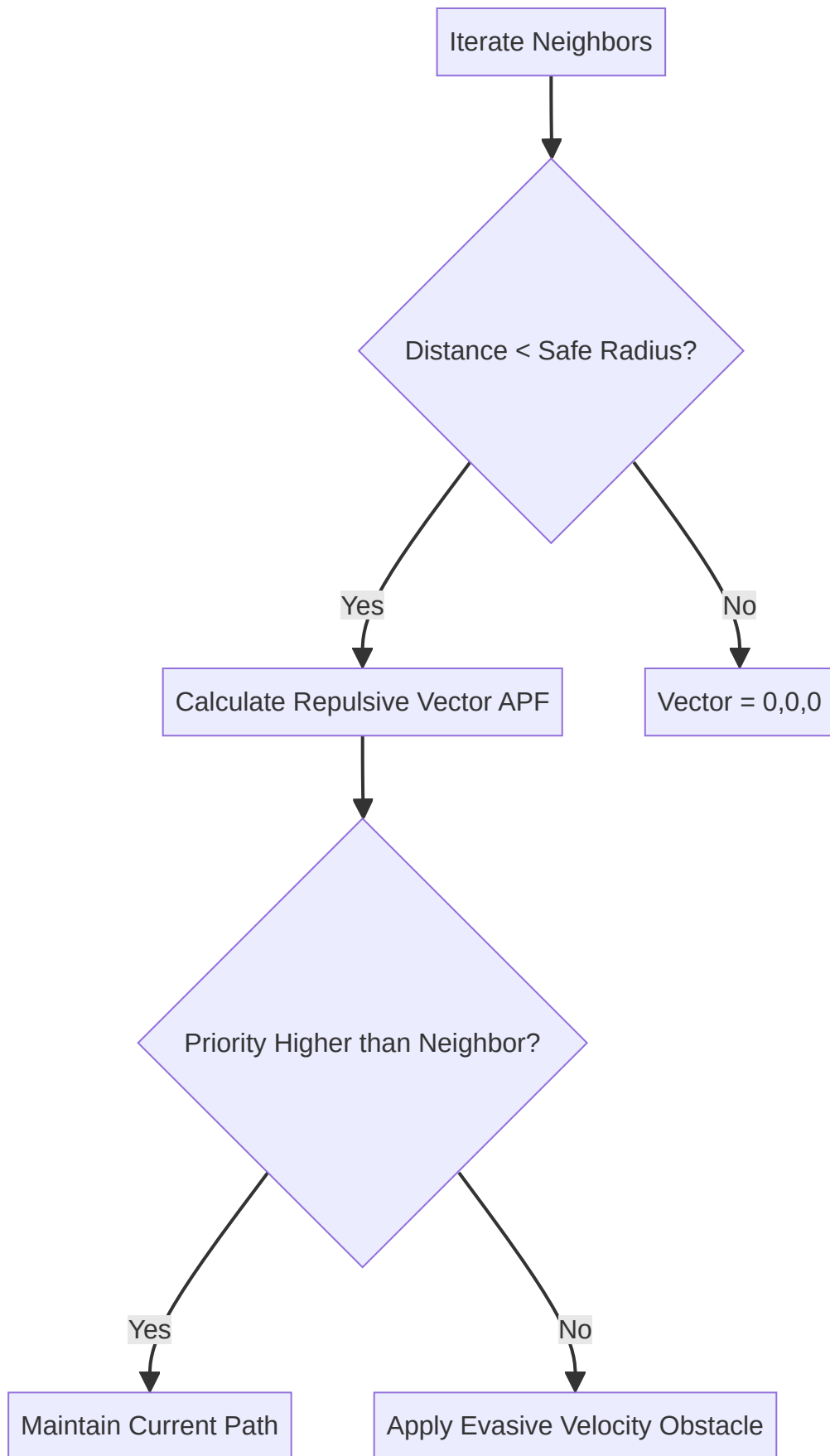




Formation Flowchart



Collision Avoidance Flowchart



The Flight Control Manager operates as a pure vector-blending pipeline. It scales vectors based on urgency (e.g., if collision is imminent, the collision vector weight approaches 1.0, nullifying the mission vector) and clamps the final output to physical hardware limits (e.g., max 5 m/s) before passing it to the Backend.

7. Advantages of the Proposed Architecture

Feature	Leader-Follower Systems	Proposed DSOS Architecture
Scalability	Poor (Leader bottleneck)	Excellent (Distributed peer-to-peer logic)
Fault Tolerance	Single Point of Failure	High (Swarm auto-recovers if any node dies)
Hardware Independence	Often coupled to specific autopilots	Abstracted via BackendFactory (PX4/AirSim)
Code Base	Different for leader vs follower	100% identical software on every drone
Sim-to-Real Gap	High effort to port	Zero effort (Config switch only)

- **No Single Point of Failure:** By ensuring all intelligence is localized and consensus-driven, taking out any arbitrary percentage of the swarm leaves the remaining drones fully operational.
 - **Backend Abstraction:** Separating MAVSDK logic into Real and Simulation backends ensures that algorithmic developers never risk physical hardware when testing complex formation physics.
-

8. Future Scope

The highly modular nature of the DSOS serves as a framework for advanced research:

- **AI Agents & Multi-Agent Reinforcement Learning (MARL):** Replacing the deterministic APF collision and formation algorithms with distributed neural network policies for dense, aggressive swarm navigation.
- **Edge AI & Federated Learning:** Utilizing the Raspberry Pi's compute (or a Jetson Nano upgrade) to run object detection at the edge, federating the model weight updates over the mesh network without transmitting raw video.
- **Digital Twin Integration:** Streaming decentralized mesh state telemetry into a cloud-based Unreal Engine environment, allowing global operators to monitor the swarm in real-time in a high-fidelity 3D replica of the physical world.
- **Massive Scale Swarms:** Optimizing the UDP heartbeat layer via spatial clustering to scale the swarm smoothly from 10s of drones to 1,000+ drones while maintaining finite local CPU usage.